UNIVERSITY OF SRI JAYEWARDENEPURA

Faculty of Technology

Department of Information and Communication Technology

ITS4243 – Microservices and Cloud Computing

Assignment 1

Name : Mathivarman. V
Index : ICT21887

Part 01

1. What is Spring Boot and why is it used?

Spring Boot is an updated version of the Spring framework that aims to make Java development much easier. The traditional Spring framework is very complex because it requires a lot of setup, many configuration files, and several manual steps before you can even start building an application. Spring Boot solves these problems by offering automatic configuration, default settings, and an embedded server. These features allow developers to quickly start and run applications without spending too much time on setup work.

One of the main reasons for using Spring Boot is that it helps developers complete projects faster and reduces the amount of repetitive or extra code. It provides built-in tools that simplify the development process and allows applications to run as stand-alone services. This means developers don't have to deploy their applications to an external server because Spring Boot can handle everything on its own. This feature makes the development process smoother and more efficient.

Spring Boot is especially useful when creating REST APIs and microservices. It is simple to use, fast to start, and includes tools that are ready for production, such as monitoring and performance tracking. Because of these features, many developers prefer using Spring Boot for modern web applications, where speed, reliability, and scalability are very important.

2. Explain the difference between Spring Framework and Spring Boot?

The Spring framework is a broad and flexible Java platform that allows developers to build large-scale enterprise applications. It offers strong features like dependency injection, web MVC, security, and data access, but it also involves a lot of manual setup. Developers must define XML or Java configurations, perform server-side setups, and manage numerous dependencies by hand.

In contrast, Spring Boot builds on the Spring framework and focuses mainly on improving development productivity. It reduces manual work by providing automatic configurations, starter dependencies, and an embedded server like Tomcat. This means you can run Spring Boot applications without installing or configuring an external server, making the process much faster and easier.

In short, Spring Framework is powerful but requires many steps to use effectively. Spring Boot, on the other hand, is a simpler, pre-configured approach that lets you build Spring applications quickly and with less effort.

3. What is Inversion of Control (IoC) and Dependency Injection (DI)?

When you build a software application, it is not responsible for creating or managing its objects on its own. Instead, this responsibility is handed over to something else that handles object creation. This idea is called Inversion of Control (IoC).

In normal programming, classes create their own objects. With IoC, the framework creates these objects and provides them whenever the application needs them. This approach reduces tight coupling between classes, making the application easier to modify, maintain, and test.

Dependency Injection (DI) is a way to implement IoC. Instead of a class creating its own dependencies, the required objects are "injected" from outside, either through a constructor, a setter, or a method. This makes the code cleaner and more flexible: the creator of the dependency does not worry about how it is made, just supplies it to the class that needs it. In short, IoC shifts control of object creation to the framework, and DI is how the framework provides the necessary objects.

4. What is the purpose of application.properties / application.yml?

This file is used to keep configuration values such as:
• Database connection information
• Server port number
• JPA/Hibernate settings
• Logging setup
• Custom application parameters

By keeping these values in one place, the program becomes easier to configure and change, instead of hardcoding them directly into the code.

5. Explain what a REST API is and list HTTP methods used?

A REST API is a way to create web services that let different systems on the internet communicate using the standard HTTP protocol. It treats data as "resources" that can be accessed and managed via URLs. Clients can request, create, update, or delete these resources, and the server responds with the data, usually in JSON format. REST APIs are popular because they are lightweight, simple to use, and work well for web and mobile applications.

Common HTTP Methods in REST:

• GET – Used to retrieve data
• POST – Used to create new data
• PUT – Used to fully update existing data

• PATCH – Used to partially update data
• DELETE – Used to remove data
• OPTIONS – Used to check which operations are supported

6. What is Spring Data JPA? What is an Entity and a Repository?

Spring Data JPA is a part of the Spring Data project that makes working with JPA much easier and faster. It eliminates the need to write complex SQL queries manually, allowing developers to interact with the database using simple Java methods. It also automatically handles common database operations like saving new records, deleting existing ones, and fetching data. This reduces boilerplate code, speeds up development, and lowers the chance of errors in database operations.

An Entity is a Java class that represents a database table in the application. Each field in the class corresponds to a column in the table, and each instance of the class represents a row in that table. Entities act as a bridge between the object-oriented code and relational database structure, enabling the application to easily store, retrieve, and manipulate data in the database without dealing directly with SQL queries. They also support relationships between tables, such as one-to-many or many-to-many, through annotations.

A Repository provides a way to access and manage data stored in Entities. By extending Spring Data JPA repository interfaces, developers automatically get common methods like save(), findById(), findAll(), and deleteById() without writing any SQL. Repositories can also include custom queries if needed. They act as the data access layer of the application, separating the database operations from business logic, making the code cleaner, easier to maintain, and testable.

7. What is the difference between @Component, @Service, @Repository, @Controller, @RestController?

1. **@Component**
   • The most general type of bean.
   • Can be applied to any class that does not fit a more specific category.
   • Serves as the parent annotation for other specialized stereotypes.

2. **@Service**
   • Used for classes containing business logic.
   • Clarifies that the class performs essential operations in the application.
   • Essentially a @Component with a more specific role.

3. **@Repository**
   • Applied to data-access classes that work with the database.

• Provides extra features, like converting database exceptions into Spring-friendly exceptions.
• Acts as a specialized @Component for data handling.

4. **@Controller**
   • Used in web applications that return HTML views.
   • Works with Spring MVC to render server-side pages.

5. **@RestController**
   • Combines @Controller and @ResponseBody.
   • Designed for REST APIs, returning JSON or XML directly instead of HTML.

8. What is @Autowired? When should we avoid it?

@Autowired is a Spring annotation that tells the framework to automatically provide the required dependency for a class. It works like giving a class a ready-made toolbox, so it doesn't have to create or look for what it needs.

Spring handles object creation and injects them exactly where they are needed, like fitting a key into a lock. This reduces repetitive code and keeps the application loosely coupled, like pieces held together by flexible threads. While @Autowired is useful, applying it directly on fields can cause problems, similar to wiring a light without checking the switch.

Field injection makes testing and maintenance harder and prevents creating immutable objects, like trying to fix a clock while its gears are moving. A better approach is **constructor injection**, where Spring provides dependencies through the class constructor. This makes it clear what a class requires, and if something is missing, the compiler immediately flags it, like a warning light on a dashboard.

9. Explain how Exception Handling works in Spring Boot (@ControllerAdvice).

Exception handling in Spring Boot allows developers to manage errors in a clean, organized, and consistent way. Instead of scattering try-catch blocks across every controller method, Spring provides a centralized mechanism to handle all application exceptions using the **@ControllerAdvice** annotation. This makes error management more structured and easier to maintain.

A class annotated with **@ControllerAdvice** acts as a global exception handler, capable of catching and processing exceptions thrown from any controller. Inside this class, you can define methods annotated with **@ExceptionHandler** to specify how different types of errors should be handled. For example, if a requested record is not found, the method can return a 404 response; if input validation fails, it can return a 400 response. You can also include custom error messages, timestamps, or any additional details in the response to make it more informative for the client.

Using this approach provides several benefits:

- Controller code stays clean and focused solely on processing requests.
- All error responses follow a consistent format across the application.
- HTTP status codes and error messages can be controlled centrally.
- It improves the clarity, readability, and maintainability of the project.
- It also makes testing easier since exception handling logic is separated from business logic.

This centralized error-handling strategy ensures that applications are robust, user-friendly, and easier to debug when issues occur.

.

10. What is the role of Maven/Gradle in a Spring Boot project?

Maven and Gradle are tools that help manage and build a Spring Boot project efficiently. In a regular Java application, you need many libraries, and managing them manually can be complicated. Maven and Gradle simplify this by automatically downloading and including all the libraries your project requires.

They also handle other important tasks like compiling your code, running tests, and packaging your application into a runnable file. Maven uses an XML file called **pom.xml**, while Gradle uses a script file, which is generally faster and more flexible. Both tools make it easier to manage dependencies and build processes without manual effort.

In short, Maven and Gradle streamline the entire application development process by taking care of all build-related tasks. This allows developers to focus on writing code, improving productivity, and ensuring the project is built consistently and correctly every time.

Part 02

GitHub Repo: https://github.com/mathivarman/Assignment01-Microservices-and-Cloud-Computing.git